

CHANDIGARH UNIVERSITY APEX INSTITUTE OF TECHNOLOGY



EXPERIMENT 8

**Subject: Advanced Database Management
System**

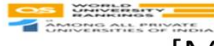
**Name: Rishav Giri
Class:24AIT-KRG-1-B
UID:24BAI70369**

Submitted To: Alok Sir

Question 1



DEPARTMENT OF CAREER
PLANNING & DEVELOPMENT



Problem Statement

[Medium Level]

A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

The company wants to provide an order report to an external client. The report must contain the **Order ID, Order Date, Customer Name, Product Name, and Final Amount**.

For every order, first calculate the **Gross Amount** using:

Gross Amount = Unit Price × Quantity

Then calculate the **Discount Amount** using:

Discount Amount = Gross Amount × Discount Percentage / 100

Finally, calculate the **Final Amount** using:

Final Amount = Gross Amount – Discount Amount

Create a **complex view** using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.

The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user **only SELECT permission on the view**. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

1. Create View

```
126 (product_id, quantity, customer_id, discount_percent, order_date)
127 VALUES
128 ('P1', 2, 'C1', 5, '2025-09-01'),
129 ('P2', 1, 'C2', 10, '2025-09-02'),
130 ('P3', 3, 'C3', 0, '2025-09-03'),
131 ('P4', 1, 'C4', 8, '2025-09-04'),
132 ('P5', 1, 'C5', 12, '2025-09-05'),
133 ('P1', 1, 'C2', 5, '2025-09-06'),
134 ('P3', 2, 'C1', 0, '2025-09-07');
135
136 create or replace view cv_employee
137 as
138 select s.order_id, s.order_date, c.full_name, p.product_name, round((p.unit_price*s.quantity)-(p.unit_price*s.quantity*s.discount_per
139 from sales_orders s
140 join product_catalog p
141 on p.product_id=s.product_id
142 join customer_master c
143 on c.customer_id=s.customer_id;
144
145 select * from cv_employee;
146
```

order_id	order_date	full_name	product_name	total_amount
1	2025-09-01	Amit Sharma	Smartphone X100	47500.00
2	2025-09-02	Priya Verma	Laptop Pro 15	58500.00
3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00
4	2025-09-04	Neha Singh	Smartwatch Fit	27600.00
5	2025-09-05	Arun Mehta	Gaming Console	39600.00
6	2025-09-06	Priya Verma	Smartphone X100	23750.00
7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00

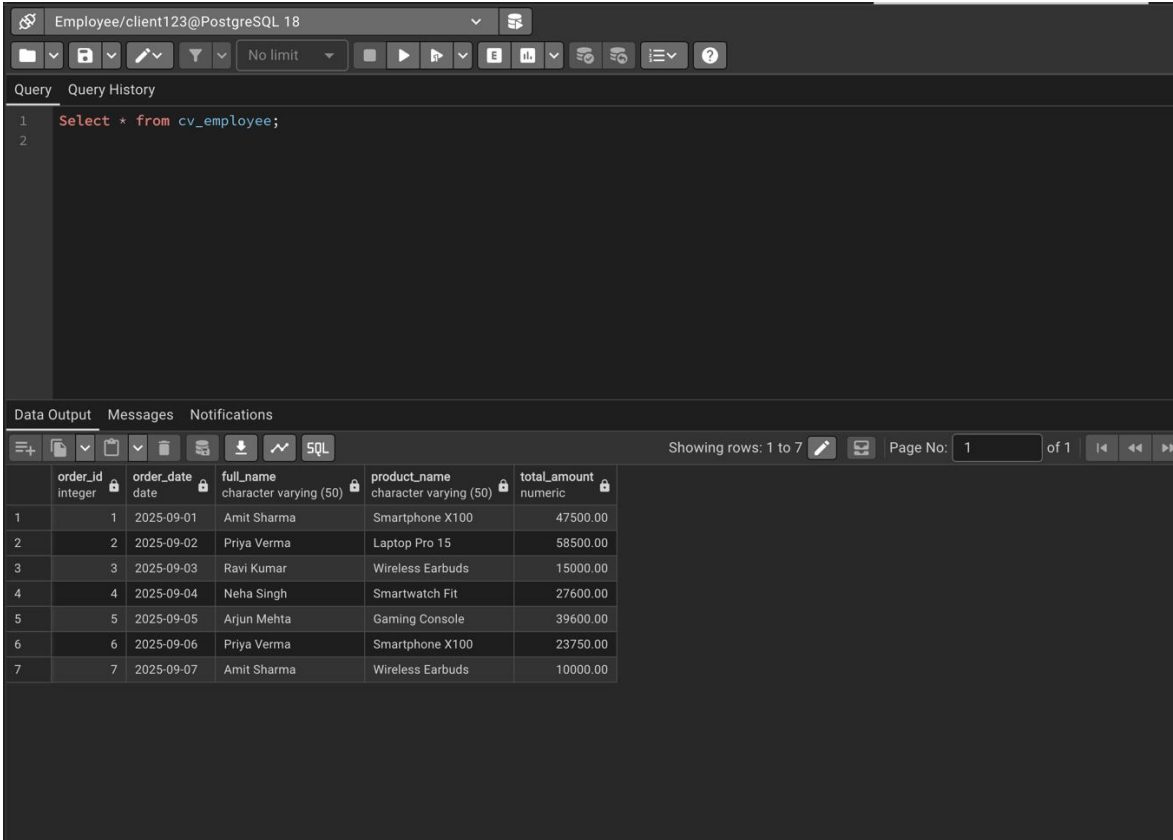
2. Create user and grant select permission

```
147
148 create user client123
149 password '12345';
150
151 grant select on cv_employee to client123;
152
```

GRANT

Query returned successfully in 62 msec.

3. Selecting from view



The screenshot shows a PostgreSQL client interface with the following components:

- Top Bar:** Displays the connection string "Employee/client123@PostgreSQL 18".
- Query Editor:** Contains the SQL query: `Select * from cv_employee;`
- Data Output Panel:** Shows the results of the query in a table format. The table has 7 rows and 6 columns: `order_id` (integer), `order_date` (date), `full_name` (character varying (50)), `product_name` (character varying (50)), and `total_amount` (numeric). The results are as follows:

	order_id	order_date	full_name	product_name	total_amount
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.00
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.00
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.00
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.00
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.00
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00

Question 2

**DEPARTMENT OF CAREER
PLANNING & DEVELOPMENT**
Department of Career Planning & Development

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY

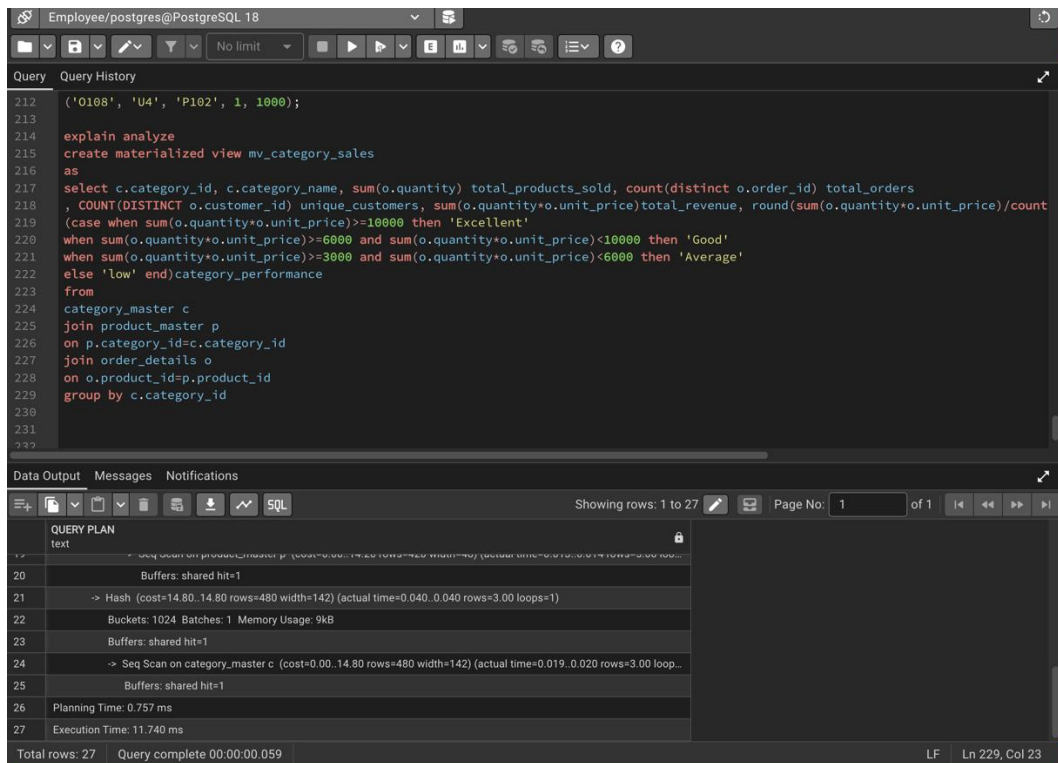
**WORLD
EDUCATION
SERVICES**
ACCREDITED ALL PRIVATE
UNIVERSITIES OF INDIA

**nirf**
RANKED #20
AMONGST TOP
UNIVERSITIES
IN INDIA

The report must contain:

- Category ID
- Category Name
- Total Products Sold
- Total Orders
- Unique Customers
- Total Revenue
- Average Revenue per Order
- Category Performance

1. Check time for execution while creating the materialized view



The screenshot shows a PostgreSQL query editor with the following SQL code:

```
212 ('0108', 'U4', 'P102', 1, 1000);
213
214 explain analyze
215 create materialized view mv_category_sales
216 as
217 select c.category_id, c.category_name, sum(o.quantity) total_products_sold, count(distinct o.order_id) total_orders
218 , COUNT(DISTINCT o.customer_id) unique_customers, sum(o.quantity*o.unit_price)total_revenue, round(sum(o.quantity*o.unit_price)/count
219 (case when sum(o.quantity*o.unit_price)>=10000 then 'Excellent'
220 when sum(o.quantity*o.unit_price)>=6000 and sum(o.quantity*o.unit_price)<10000 then 'Good'
221 when sum(o.quantity*o.unit_price)>=3000 and sum(o.quantity*o.unit_price)<6000 then 'Average'
222 else 'low' end)category_performance
223 from
224 category_master c
225 join product_master p
226 on p.category_id=c.category_id
227 join order_details o
228 on o.product_id=p.product_id
229 group by c.category_id
230
231
232
```

The query is executed, and the execution plan is shown below:

QUERY PLAN	text
20	Buffers: shared hit=1
21	-> Hash (cost=14.80..14.80 rows=480 width=142) (actual time=0.040..0.040 rows=3.00 loops=1)
22	Buckets: 1024 Batches: 1 Memory Usage: 9kB
23	Buffers: shared hit=1
24	-> Seq Scan on category_master c (cost=0.00..14.80 rows=480 width=142) (actual time=0.019..0.020 rows=3.00 loop...
25	Buffers: shared hit=1
26	Planning Time: 0.757 ms
27	Execution Time: 11.740 ms

Total rows: 27 Query complete 00:00:00.059 LF Ln 229, Col 23

2. Execute the materialized view

The screenshot shows a database query editor with a SQL query and its output. The query is as follows:

```
as
select c.category_id, c.category_name, sum(o.quantity) total_products_sold, count(distinct o.order_id) total_orders
, COUNT(DISTINCT o.customer_id) unique_customers, sum(o.quantity*o.unit_price)total_revenue, round(sum(o.quantity*o.unit_price)/count
(case when sum(o.quantity*o.unit_price)>=10000 then 'Excellent'
when sum(o.quantity*o.unit_price)>=6000 and sum(o.quantity*o.unit_price)<10000 then 'Good'
when sum(o.quantity*o.unit_price)>=3000 and sum(o.quantity*o.unit_price)<6000 then 'Average'
else 'low' end)category_performance
from
category_master c
join product_master p
on p.category_id=c.category_id
join order_details o
on o.product_id=p.product_id
group by c.category_id;
```

The output shows the following data:

category_id	category_name	total_products_sold	total_orders	unique_customers	total_revenue	avg_revenue_per_order	category_performance
C1	Electronics	5	4	3	13000.00	3250.00	Excellent
C2	Clothing	4	3	2	10000.00	3333.33	Excellent
C3	Books	3	1	1	1500.00	1500.00	low

3. An

The screenshot shows a database query editor with a SQL query and its output. The query is as follows:

```
as
select c.category_id, c.category_name, sum(o.quantity) total_products_sold, count(distinct o.order_id) total_orders
, COUNT(DISTINCT o.customer_id) unique_customers, sum(o.quantity*o.unit_price)total_revenue, round(sum(o.quantity*o.unit_price)/count
(case when sum(o.quantity*o.unit_price)>=10000 then 'Excellent'
when sum(o.quantity*o.unit_price)>=6000 and sum(o.quantity*o.unit_price)<10000 then 'Good'
when sum(o.quantity*o.unit_price)>=3000 and sum(o.quantity*o.unit_price)<6000 then 'Average'
else 'low' end)category_performance
from
category_master c
join product_master p
on p.category_id=c.category_id
join order_details o
on o.product_id=p.product_id
group by c.category_id;
```

The output shows the following data:

category_id	category_name	total_products_sold	total_orders	unique_customers	total_revenue	avg_revenue_per_order	category_performance
C1	Electronics	5	4	3	13000.00	3250.00	Excellent
C2	Clothing	4	3	2	10000.00	3333.33	Excellent
C3	Books	3	1	1	1500.00	1500.00	low